

# Lab 2 — Reverse Engineering hack\_app with GDB & Friends

**Student:** Melnikov Sergei ([s.melnikov@innopolis.university](mailto:s.melnikov@innopolis.university)) **Tools used:** readelf, ldd, strings, objdump (static analysis), strace (dynamic tracing), GDB (debugging), Python 3  
**Target:** hack\_app (x86-64 ELF, PIE, compiled with GCC 9.4.0)  
**Artifacts delivered:** keygen (keygen.py, keygen.c), binary patch (patch.py → hack\_app\_patched)

## 1. Reconnaissance

### 1.1 ELF Header (readelf -h hack\_app)

```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                               2's complement, little endian
  Type:                               DYN (Position-Independent Executable file)
  Machine:                             Advanced Micro Devices X86-64
  Entry point address:                 0x1200
```

The binary is a 64-bit PIE executable, not stripped (full .symtab present).

### 1.2 Shared Libraries (ldd hack\_app)

```
libcrypto.so.1.1 => /lib/x86_64-linux-gnu/libcrypto.so.1.1
libc.so.6        => /lib/x86_64-linux-gnu/libc.so.6
libdl.so.2       => /lib/x86_64-linux-gnu/libdl.so.2
libpthread.so.0  => /lib/x86_64-linux-gnu/libpthread.so.0
```

Key finding: **libcrypto.so.1.1** (OpenSSL) — the app uses cryptographic functions.

### 1.3 Strings Analysis (strings hack\_app)

```
MD5_Final / MD5_Init / MD5_Update      ← OpenSSL MD5 API
%08X%08X                               ← HWID format
%02x                                     ← hex encoding format
/proc/self/exe                         ← binary self-path
user.license                           ← xattr name for license storage
Welcome to Lab2 super secure program!
Your HWID is %08X%08X.
Enter the license key:
Now you app is activated! Thanks for purchasing!
Provided key is wrong! App is closing!
Your app is licensed to this PC!
```

Key findings:

- License is stored as **extended attribute** user.license on the binary itself (not a file on disk)
- Hardware ID is derived from **two 32-bit values** formatted as %08X%08X
- License validation uses **MD5**

### 1.4 Symbol Table (readelf -s hack\_app , excerpt)

```
49: 0000000000004040 4096 OBJECT GLOBAL DEFAULT 26 xattrValue
60: 0000000000005040 33 OBJECT GLOBAL DEFAULT 26 md5decode
61: 0000000000005070 16 OBJECT GLOBAL DEFAULT 26 md5digest
75: 0000000000001397 77 FUNC GLOBAL DEFAULT 16 calc_md5
78: 0000000000005080 4096 OBJECT GLOBAL DEFAULT 26 binaryPath
82: 0000000000006080 33 OBJECT GLOBAL DEFAULT 26 userInput
84: 00000000000060b0 17 OBJECT GLOBAL DEFAULT 26 PSN
87: 00000000000013e4 660 FUNC GLOBAL DEFAULT 16 main
```

## 2. Static Analysis (objdump)

Full disassembly of `main` (at `0x13e4`, 660 bytes) was obtained via `objdump -d hack_app`. The reconstructed C pseudocode:

```
int main() {
    unsigned int eax, ebx, ecx, edx;
    int licensed = 0;

    // Step 1: Get CPU identifier via CPUID leaf 1
    __get_cpuid(1, &eax, &ebx, &ecx, &edx);

    // Step 2: Byte-swap EAX and EDX
    unsigned int part1 = bswap32(eax); // from -0x2c(rbp)
    unsigned int part2 = bswap32(edx); // from -0x20(rbp)

    // Step 3: Build 16-char hex HWID string
    char PSN[17];
    snprintf(PSN, 17, "%08X%08X", part1, part2);

    // Step 4: Compute MD5 of PSN
    calc_md5(PSN, 16); // result in global md5digest[16]

    // Step 5: Hex-encode reversed digest → license key
    char md5decode[33];
    for (int i = 0; i < 16; i++)
        sprintf(md5decode + i*2, "%02x", md5digest[15 - i]);

    // Step 6: Resolve own binary path
    readlink("/proc/self/exe", binaryPath, 0x1000);

    // Step 7: Read stored license from xattr
    getxattr(binaryPath, "user.license", xattrValue, 0x1000);

    puts("Welcome to Lab2 super secure program!");

    // Step 8: Compare stored license with computed key
    if (strcmp(md5decode, xattrValue, 33) == 0)
        licensed = 1;

    if (licensed == 0) {
        printf("Your HWID is %08X%08X.\n", part1, part2);
        scanf("%32s", userInput);
        if (strcmp(md5decode, userInput, 33) == 0) {
            setxattr(binaryPath, "user.license", md5decode, 33, 0);
            puts("Now you app is activated! Thanks for purchasing!");
        } else {
            puts("Provided key is wrong! App is closing!");
        }
    } else {
        puts("Your app is licensed to this PC!");
    }
    system("read -p 'Press Enter to continue...' var");
    return 0;
}
```

### License Algorithm Summary

```
CPUID(leaf=1) → EAX, EDX
↓
bswap32(EAX), bswap32(EDX)
↓
PSN = sprintf("%08X%08X", bswap_eax, bswap_edx)    [16-char hex string]
↓
digest = MD5(PSN)                                [16 raw bytes]
↓
license = hex(digest[15], digest[14], ..., digest[0]) [32-char hex string]
```

## The License Decision Point

Original (objdump -d hack\_app , around VA 0x1597):

```
1597: e8 b4 fb ff ff    call    strncmp@plt      ; compare xattr vs computed key
159c: 85 c0             test    eax, eax
159e: 75 07             jne     15a7             ; ← skip "licensed=1" if mismatch
15a0: c7 45 e4 01 ..    movl    $0x1,-0x1c(rbp) ; licensed = 1
15a7: 83 7d e4 00       cmpl    $0x0,-0x1c(rbp)
15ab: 0f 85 8e 00 ..    jne     163f             ; → "Your app is licensed" path
```

---

## 3. Artifact 1 – Keygen

---

Source: `keygen.py`

```
#!/usr/bin/env python3
"""Keygen for hack_app – reproduces the CPUID→MD5 license algorithm."""
import ctypes, hashlib, mmap, struct

def cpuid_via_asm(leaf):
    code = bytes([
        0x53,                # push  rbx
        0x89, 0xf0,          # mov   eax, esi
        0x31, 0xc9,          # xor   ecx, ecx
        0x0f, 0xa2,          # cpuid
        0x89, 0x07,          # mov   [rdi],  eax
        0x89, 0x5f, 0x04,     # mov   [rdi+4], ebx
        0x89, 0x4f, 0x08,     # mov   [rdi+8], ecx
        0x89, 0x57, 0x0c,     # mov   [rdi+12], edx
        0x5b,                # pop   rbx
        0xc3,                # ret
    ])
    buf = mmap.mmap(-1, 4096,
                    prot=mmap.PROT_READ | mmap.PROT_WRITE | mmap.PROT_EXEC)
    buf.write(code)
    FUNC = ctypes.CFUNCTYPE(None, ctypes.POINTER(ctypes.c_uint32), ctypes.c_uint32)
    func = FUNC(ctypes.addressof(ctypes.c_char.from_buffer(buf)))
    result = (ctypes.c_uint32 * 4)()
    func(result, leaf)
    buf.close()
    return result[0], result[1], result[2], result[3]

def bswap32(v):
    return struct.unpack(">I", struct.pack("<I", v & 0xffffffff))[0]

def generate_license():
    eax, ebx, ecx, edx = cpuid_via_asm(1)
    part1, part2 = bswap32(eax), bswap32(edx)
    PSN = "%08X%08X" % (part1, part2)
    digest = hashlib.md5(PSN.encode()).digest()
    return PSN, "".join("%02x" % digest[15 - i] for i in range(16))

hwid, key = generate_license()
print(f"HWID:    {hwid}")
print(f"License: {key}")
```

A C version ( `keygen.c` ) is also provided, using `<cpuid.h>` and OpenSSL's `MD5_*` API directly.

## Verification — keygen output fed to original binary

```
$ python3 keygen.py
HWID:    C0060200158B8B0F
License: c028fdefde9ccc6d8d10f7eebda47a4b

$ echo "c028fdefde9ccc6d8d10f7eebda47a4b" | ./hack_app
Welcome to Lab2 super secure program!
Your HWID is C0060200158B8B0F.
Enter the license key: Now you app is activated! Thanks for purchasing!
```

After activation, the license is persisted in `xattr` and subsequent runs print:

```
Welcome to Lab2 super secure program!
Your app is licensed to this PC!
```

## 4. Artifact 2 — Binary Patch

## Approach

NOP out the conditional branch at VA `0x159e` that guards `licensed = 1`. This causes the app to always set `licensed = 1`, bypassing both the xattr check and the user input prompt.

| Offset | Original       | Patched         | Meaning                                        |
|--------|----------------|-----------------|------------------------------------------------|
| 0x159e | 75 07 (JNE +7) | 90 90 (NOP NOP) | Always fall through to <code>licensed=1</code> |

Source: `patch.py`

```
#!/usr/bin/env python3
"""Binary patch: NOP the JNE at 0x159e to always set licensed=1."""
import shutil, sys, os

SRC, DST = "hack_app", "hack_app_patched"
FILE_OFFSET = 0x159e
ORIGINAL = bytes([0x75, 0x07]) # jne +7
PATCHED = bytes([0x90, 0x90]) # nop; nop

shutil.copy2(SRC, DST)
with open(DST, "r+b") as f:
    f.seek(FILE_OFFSET)
    assert f.read(2) == ORIGINAL, "Unexpected bytes at patch site"
    f.seek(FILE_OFFSET)
    f.write(PATCHED)
os.chmod(DST, 0o755)
print(f"Patched {DST}: JNE → NOP NOP at offset 0x{FILE_OFFSET:x}")
```

## Disassembly comparison

Before (original):

```
159c: 85 c0      test    eax, eax
159e: 75 07      jne     15a7          ; skip licensed=1 if key mismatch
15a0: c7 45 e4 01 movl    $0x1,-0x1c(rbp)
```

After (patched):

```
159c: 85 c0      test    eax, eax
159e: 90         nop                     ; ← was JNE
159f: 90         nop
15a0: c7 45 e4 01 movl    $0x1,-0x1c(rbp) ; always executes now
```

## Verification — patched binary with no license

```
$ python3 patch.py
Patched hack_app_patched: NOP-ed JNE at offset 0x159e

$ echo "" | ./hack_app_patched
Welcome to Lab2 super secure program!
Your app is licensed to this PC!
```

The patched binary reports "licensed" without any license key or xattr present.